HDOC Day-61

Finding Two Sum Indices in an Array Using Functions and Pointers in C

Aim

To write a C program using a variable-length integer array, user-defined functions, pointers, and nested loops to find two distinct indices whose elements add up to a given target. The program prints the zero-based indices separated by a space, or **-1 -1** when no valid pair exists.

Algorithm

1. Start the program and read the array size.
2. Check that the size is positive; otherwise display an error and terminate.
3. Declare the variable-length array nums[size] and read its positive integer elements.
4. Read the target integer.
5. Pass the array, size, and target to two_sum() using the array address.
6. Use an outer loop to select index i and an inner loop beginning at i + 1 to select index j.
7. Compare nums[i] + nums[j] with target. If equal, print i and j and return immediately.
8. If all pairs have been examined without a match, print -1 -1.
9. Stop the program.

C Program

```c
#include <stdio.h>

void two_sum(int *nums, int size, int target);

int main()
{
    int size, target, i;

    printf("Enter the size of array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    int nums[size];

    printf("Enter %d positive array elements: ", size);
    for (i = 0; i < size; i++)
    {
        scanf("%d", &nums[i]);
        if (nums[i] <= 0)
        {
            printf("Invalid input! Enter positive integers only.\n");
            return 0;
        }
    }

    printf("Enter the target: ");
    scanf("%d", &target);
    two_sum(nums, size, target);
    return 0;
}

/* Find two distinct indices whose elements sum to target */
void two_sum(int *nums, int size, int target)
{
    int i, j;
    for (i = 0; i < size - 1; i++)
    {
        for (j = i + 1; j < size; j++)
        {
            if (*(nums + i) + *(nums + j) == target)
            {
                printf("Two Sum Indices: %d %d\n", i, j);
                return;
            }
        }
    }
    printf("Two Sum Indices: -1 -1\n");
}
```

Explanation

The function **two_sum(int *nums, int size, int target)** receives the array through a pointer. The expressions ***(nums + i)** and ***(nums + j)** access the elements at indices i and j. Beginning the inner loop at i + 1 guarantees distinct indices. The function takes $O(n^2)$ time and O(1) auxiliary space.

Example: For nums = [2, 7, 11, 15] and target = 9, the first pair is 2 + 7 = 9, at indices 0 and 1. The output is **Two Sum Indices: 0 1**.

Test Case Table

Test	Input Array	Target	Expected Output
1	2 7 11 15	9	0 1
2	3 2 4	6	1 2
3	3 3	6	0 1
4	1 2 3 4	10	-1 -1
5	5 10 15 20	25	0 3
6	1 4 6 8	7	0 2
7	10 20 30	50	1 2
8	7	14	-1 -1
9	1 2 3 4	3	0 1
10	size = 0	-	Invalid array size

Sample Outputs

Valid Pair

```
Enter the size of array: 4
Enter 4 positive array elements: 2 7 11 15
Enter the target: 9
Two Sum Indices: 0 1
```

Duplicate Elements

```
Enter the size of array: 2
Enter 2 positive array elements: 3 3
Enter the target: 6
Two Sum Indices: 0 1
```

No Valid Pair

```
Enter the size of array: 4
Enter 4 positive array elements: 1 2 3 4
Enter the target: 10
Two Sum Indices: -1 -1
```

Single Element

```
Enter the size of array: 1
Enter 1 positive array elements: 7
Enter the target: 14
Two Sum Indices: -1 -1
```

Invalid Array Size

```
Enter the size of array: 0
Invalid array size!
```

Result

The C program uses variable-length arrays, pointers, user-defined functions, nested loops, and conditional statements to find two distinct zero-based indices whose elements sum to the target. It displays the indices when a pair exists and -1 -1 otherwise, as required.